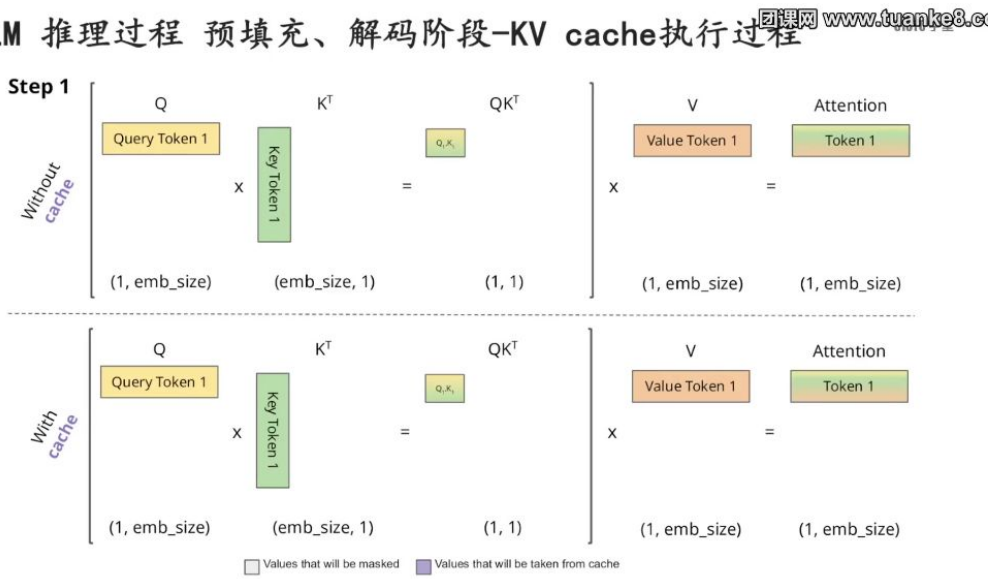
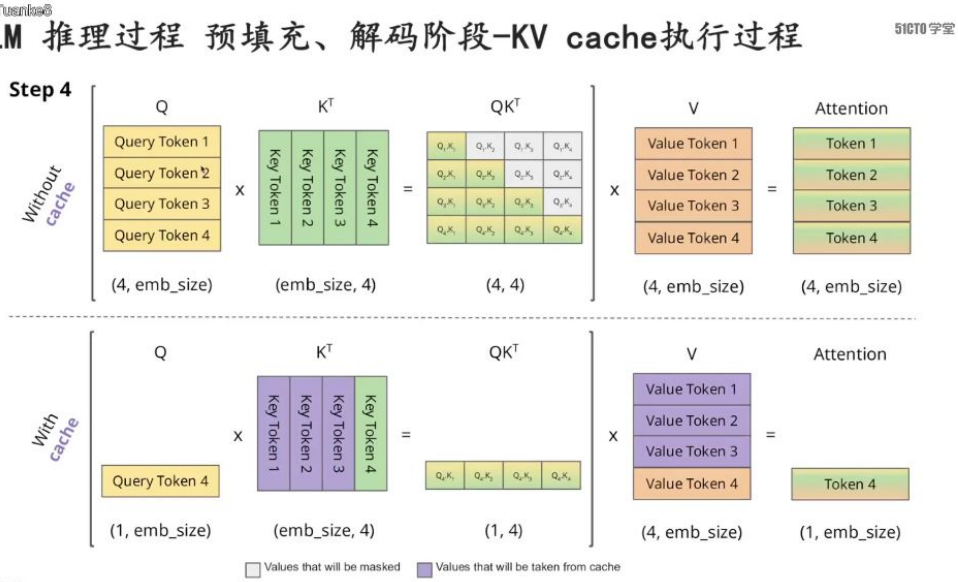
以下为AI生成的图文笔记的内容

一、KV Cache执行过程 00:01

1. 例题：新年快乐kv cache示例 00:10



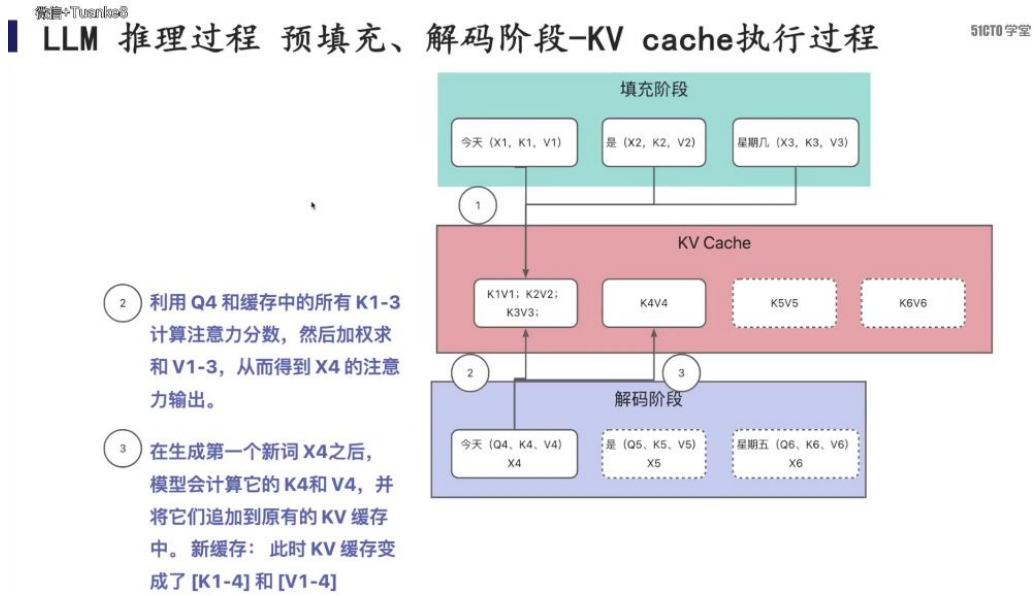
- **无缓存机制**：每次新token输入时，需要重新计算所有Key和Value向量，计算复杂度为 $O(n^2)$
- **有缓存机制**：紫色部分显示已缓存的KV向量，只需计算新token的Query向量，复用历史KV向量
- **计算优势**：黄色部分只需生成新token的Q向量，K/V向量沿用缓存，显著减少计算量



- **递推过程**：生成第4个token时，只需计算Q_4与缓存中的$K_{1\text{-}3}$的注意力分数，加权求和 $V_{1\text{-}3}$
- **缓存更新**：新生成的K_4/V_4会追加到缓存中，供后续token计算使用

2. 预填充阶段和解码阶段kv cache过程 02:37

微信:Tuanke8



- **预填充阶段**：
  - **输入处理**：将提示词"今天是星期几"的每个token（X_1-X_3）对应的K_1/V_1-K_3/V_3存入缓存
  - **缓存初始化**：建立初始KV缓存[K1-3]和[V1-3]
- **解码阶段**：
  - **首词生成**：用Q_4与缓存中的K_{1-3}计算注意力分数，加权V_{1-3}生成"今天"(X_4)
  - **缓存扩展**：将新生成的K_4/V_4追加到缓存，形成[K1-4]和[V1-4]
  - **迭代过程**：后续每个新token都会复用全部历史KV缓存，并扩展自身KV向量
- **瓶颈分析**：
  - **空间增长**：提示词长度和生成内容长度直接影响缓存大小，呈线性增长关系
  - **计算特性**：自回归架构要求每个新token必须与所有历史token计算注意力关系

二、kv cache缓存的优化 05:25

- **优化必要性**：
  - **双重占用**：预填充阶段和解码阶段都会持续占用缓存空间
  - **性能关键**：transformer推理过程中KV缓存访问频率极高，成为性能瓶颈
- **优化方向**：
  - **缓存压缩**：对历史KV向量进行有损/无损压缩
  - **访问优化**：改进缓存数据结构提高访问效率
  - **更新策略**：设计智能的KV向量淘汰机制
- **典型案例**：
  - **长文本处理**：当处理超长提示词时，需要特殊优化策略避免缓存溢出
  - **持续对话**：多轮对话场景下缓存管理策略直接影响系统响应速度

三、知识小结

知识点	核心内容	易混淆点/关键细节	重要性
KV Cache的作用	在大模型推理中缓存已计算的Key-Value向量，避免重复计算	KV Cache仅缓存K/V向量，**Q向量需实时生成**	★★★★
预填充阶段	处理提示词（如"今天是星期几"）时，将每个token的K/V存入缓存	提示词越长，**KV Cache占用越大**	★★★

解码阶段	自回归生成新token时，**复用缓存的K/V**，仅计算新token的Q	新token的K/V会**动态追加到缓存**	★★★★
KV Cache瓶颈	缓存随文本长度**线性增长**，影响内存和计算效率	填充阶段和解码阶段**双重占用缓存**	★★★★★
优化方向	缓存压缩技术、内存管理策略	需平衡**计算效率**与**缓存精度**	★★★★
注意力计算过程	Q与历史K计算权重，再加权求和历史V	**紫色部分（缓存）** vs **黄色部分（新计算）**	★★★